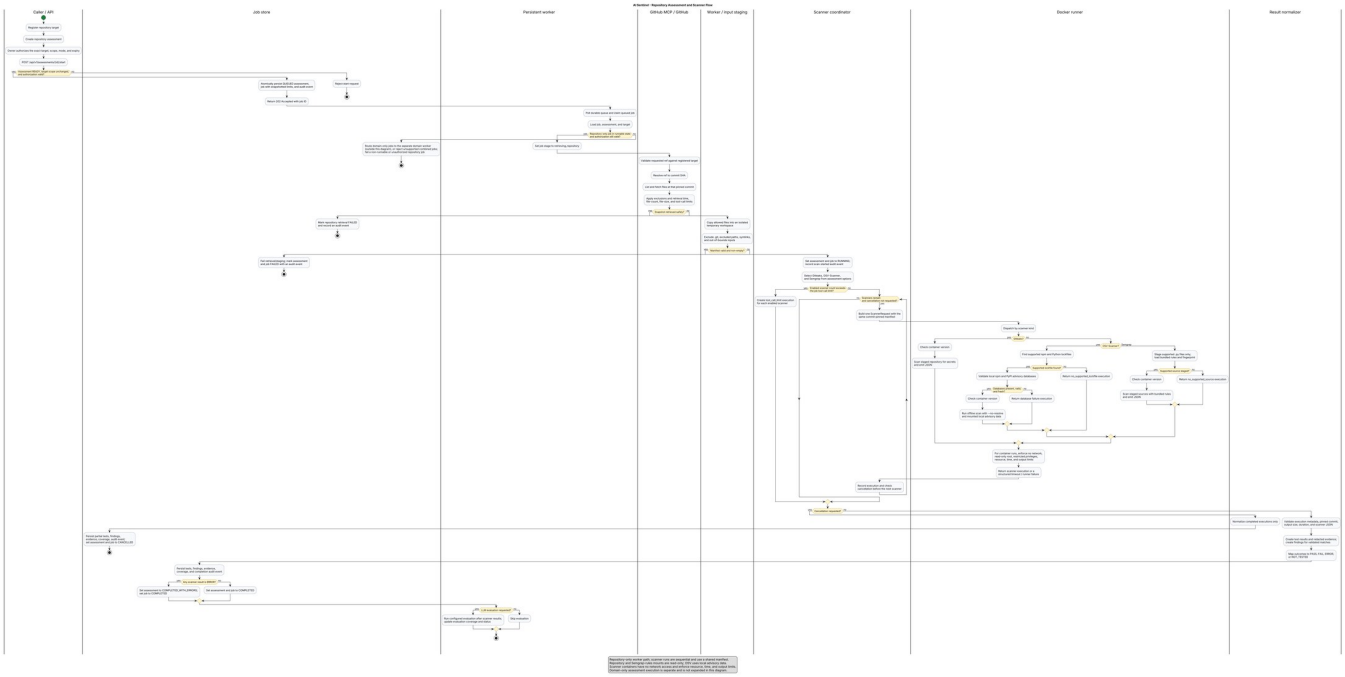
AI Sentinel Repository Scan

Swimlane Node-by-Node Reference



A source-grounded explanation of all 63 activity and decision nodes in the supplied PNG, across eight swimlanes. Each entry identifies code paths, methods and external tools, data handled, and branch behavior.

Scope: repository-only assessment flow. The image is included here for reference; the separate domain-only worker is noted but not expanded.

What this document covers

This node-by-node reference is based on `REPOSITORY_SCAN_SWIMLANE.png` in this project. It follows the repository-only path and describes the implementation behind every activity and decision shown in the swimlane. Node numbers below are for this document; they are not IDs embedded in the diagram. It covers 63 activity and decision nodes across the diagram's eight swimlanes.

The flow begins when an authorized owner queues an assessment. A separate persistent worker retrieves a commit-pinned repository snapshot, prepares a bounded staging workspace, runs enabled scanners one at a time in restricted Docker containers, normalizes their JSON, and persists tests, findings, evidence, coverage, audit events, and final status.

The diagram deliberately routes domain-only work to a separate worker and rejects combined repository/domain scans in this worker. It does not expand the separate domain-check flow.

Swimlane: Caller / API

1. Register repository target

- Implementation: `create_target()` in `src/ai_sentinel/api/routes/targets.py`; `TargetService.create()` in `src/ai_sentinel/targets/service.py`.
- Methods/tools: FastAPI `POST /api/v1/targets`; `normalize_target_scope()`; repository persistence and `AuditService.build_event()`.
- Inputs and work: Authenticated principal, repository coordinates, name, environment, and proposed scope. The service normalizes scope and creates a target owned by that principal and tenant.
- Output / rule: A target record plus `target.created` audit event. Registering a target proposes scope; it does not authorize a scan.

2. Create repository assessment

- Implementation: `create_assessment()` in `src/ai_sentinel/api/routes/assessments.py`; `AssessmentService.create()` in `src/ai_sentinel/assessments/service.py`.
- Methods/tools: FastAPI `POST /api/v1/assessments`; visible-target and ownership checks; scope validation; quota check; repository write with audit.
- Inputs and work: Target ID, repository-only mode, requested scan options, repository ref, and exclusions. The service snapshots target scope and initializes requested-control coverage.
- Output / rule: A `DRAFT` assessment and `assessment.created` event.

3. Owner authorizes exact target, scope, mode, and expiry

- Implementation: `authorize_assessment()` in `src/ai_sentinel/api/routes/assessments.py`; `AssessmentService.authorize()` in `src/ai_sentinel/assessments/service.py`; authorization helpers in `src/ai_sentinel/policy/authorization.py`.
- Methods/tools: FastAPI `POST /api/v1/assessments/{id}/authorize`; `create_authorization_record()`; `validate_authorization()`; `AuditService`.
- Inputs and work: Authenticated principal and owner-attestation request. The service derives the approver identity from the principal and checks the target scope has not changed.
- Output / rule: Assessment becomes `READY`; authorization is bound to the assessment, target, mode, scope, and expiry. The authorization transition is audited. Client-supplied approver identity is not trusted.

4. POST `/api/v1/assessments/{id}/start`

- Implementation: `start_assessment()` in `src/ai_sentinel/api/routes/assessments.py`.
- Methods/tools: FastAPI route delegates to `AssessmentService.start()`.
- Inputs and work: Assessment ID and authenticated principal.
- Output / rule: A durable job is queued and a `202 Accepted` response returns the job. The API does not execute scanners inline.

5. Decision: assessment READY, target scope unchanged, and authorization valid?

- Implementation: `AssessmentService.start()` in `src/ai_sentinel/assessments/service.py`; authorization policy helper.
- Methods/tools: `get_visible()`, `_target()`, target scope snapshot comparison, `validate_authorization()`, and concurrency checks.
- Yes branch: Continue to node 6.
- No branch: Node 8 rejects the start request. Non-READY state, changed scope, missing/invalid authorization, or concurrency conflicts prevent queuing.

6. Atomically persist QUEUED assessment, job with snapshotted limits, and audit event

- Implementation: `AssessmentService.start()`; `LocalDurableJobQueue.prepare_enqueue()` in `src/ai_sentinel/jobs/queue.py`; repository implementations in `src/ai_sentinel/db/json_repository.py` and `src/ai_sentinel/db/sql_repository.py`.
- Methods/tools: `Repository.write_records_with_audit()`; `RecordOperation`; durable JSON or SQL-backed persistence; `assessment.queued` audit event.
- Data: New `Job` has status `QUEUED`, stage `queued`, progress `0`, owner and tenant IDs, and a snapshot of duration, tool-call, retry, output, workflow-step, and LLM budgets from settings.
- Output / rule: Assessment and job transition together. Persistence conflicts are handled as conflicts, not as successful queue operations.

7. Return 202 Accepted with job ID

- Implementation: Route response from `start_assessment()`.
- Methods/tools: FastAPI response model `Job`, HTTP 202.
- Output: Caller can poll the job and assessment status while the separate worker processes it.

8. Reject start request

- Implementation: `AssessmentService.start()` raises `AssessmentServiceError`; route converts it to an HTTP error.
- Methods/tools: State, ownership, scope, authorization, quota/concurrency, and persistence-conflict checks.
- Output: No accepted job is returned for a request that fails the start preconditions.

Swimlane: Persistent worker

9. Poll durable queue and claim queued job

- Implementation: `main()` in `src/ai_sentinel/jobs/worker_cli.py`; `watch_queue()` and `process_queued_jobs()` in `src/ai_sentinel/jobs/persistent_worker.py`.
- Methods/tools: Worker CLI `--watch`; queue polling; optional stale-job recovery; repository `claim_job()` to avoid duplicate execution; heartbeat while watching.
- Input / output: Finds queued jobs in creation order and claims one before passing its ID to `_execute_job()`.

10. Load job, assessment, and target

- Implementation: `_execute_job()` in `src/ai_sentinel/jobs/worker_cli.py`; repository lookup helpers in `RepositoryScanWorker`.
- Methods/tools: `Repository.get_record()` over JOBS, ASSESSMENTS, and TARGETS; typed `Job`, `Assessment`, and `Target` schemas.
- Output / rule: Missing records or incompatible job/assessment state are explicit worker errors.

11. Decision: repository-only job runnable and authorization still valid?

- Implementation: `_execute_job()` and `GitHubRepositoryScanWorker.execute()` in `src/ai_sentinel/scanners/service.py`; `_validate_authorization()`.
- Methods/tools: Mode and run-state checks, ownership and target scope linkage, `validate_authorization()`.
- Yes branch: Begin repository retrieval at node 13.
- No branch: Domain-only jobs are routed to their separate worker; combined repository/domain jobs and non-runnable or unauthorized repository jobs are rejected at node 12.

12. Route domain-only separately, reject unsupported combined or invalid jobs

- Implementation: `_execute_job()` in `src/ai_sentinel/jobs/worker_cli.py`; `PassiveDomainCheckWorker` is separate.
- Methods/tools: `AssessmentMode.DOMAIN_ONLY` branch; `run_domain_checks` check; `ScannerWorkerError` / `ValueError` surfaced by the worker.
- Output / rule: Domain-only flow is outside this diagram. This worker explicitly rejects combined repository and domain scans. Invalid repository state/authorization does not proceed to retrieval.

Swimlane: GitHub MCP / GitHub

13. Set job stage to retrieving_repository

- Implementation: `GitHubRepositoryScanWorker.execute()` in `src/ai_sentinel/scanners/service.py`.
- Methods/tools: `Repository.write_records_with_audit()` updates job stage to `retrieving_repository`, progress to 1, and assessment/job to `RUNNING`; writes `assessment.repository_retrieval_started`.
- Output: A persisted, auditable retrieval stage before external file reads.

14. Validate requested ref against registered target

- Implementation: `GitHubMcpAdapter.fetch_repository()` in `src/ai_sentinel/github_mcp/adapter.py`.
- Methods/tools: Compare `requested_ref` with registered `repository.ref`; bounded `asyncio.timeout`; MCP client session and tool allowlist validation.
- Failure: A mismatched ref or unavailable/invalid MCP operation raises `GitHubMcpError`, leading to the retrieval-failure path.

15. Resolve ref to a commit SHA

- Implementation: `_resolve_commit()` in `src/ai_sentinel/github_mcp/adapter.py`.
- Methods/tools: GitHub MCP `get_commit` tool through `_call()`; arguments include owner, repo, requested SHA/ref, and `detail: none`.
- Validation: Returned SHA must match the 40- or 64-character commit pattern; returned repository coordinates are checked.
- Output: Immutable `commit_sha` used for all subsequent file reads.

16. List and fetch files at that pinned commit

- Implementation: `_fetch_files()` in `src/ai_sentinel/github_mcp/adapter.py`.
- Methods/tools: GitHub MCP `get_file_contents`; directories are traversed from the root, then text files are fetched at the resolved SHA.
- Validation: Repository-relative paths, allowed entry types, no duplicate paths, metadata/content SHA and size consistency, and no out-of-scope paths.
- Output: Files staged under a temporary source directory at one commit.

17. Apply exclusions and retrieval time/file/tool-call limits

- Implementation: `fetch_repository()` and `_fetch_files()`; `_CallBudget`; `RepositoryReadLimits` supplied by `GitHubRepositoryScanWorker.execute()`.
- Methods/tools: Excluded-path normalization, per-file and total byte bounds, max file count, timeout context, and MCP call budget.
- Output / rule: Excluded paths are omitted; exceeding any retrieval limit safely fails retrieval rather than returning a partial success.

18. Decision: snapshot retrieved safely?

- Implementation: `fetch_repository()` converts timeout and unexpected MCP errors to `GitHubMcpError`; worker handles it in `GitHubRepositoryScanWorker.execute()`.
- Yes branch: Continue to isolated workspace staging at node 20.
- No branch: Node 19 marks retrieval failed and records an audit event.

19. Mark repository retrieval FAILED and record audit event

- Implementation: `_fail_retrieval()` in `src/ai_sentinel/scanners/service.py`.
- Methods/tools: Re-read current assessment/job, guard against changed state or cancellation, then `write_records_with_audit()`.
- Output: Assessment/job become `FAILED`, stage becomes `repository_retrieval_failed`, and `assessment.repository_retrieval_failed` is recorded. Cancellation prevents overwriting the cancellation state.

Swimlane: Worker / input staging

20. Copy allowed files into an isolated temporary workspace

- Implementation: `GitHubRepositoryScanWorker.execute()` creates a temporary source root; `RepositoryScanWorker.execute()` creates a separate temporary scan workspace; `build_input_manifest()` in `src/ai_sentinel/scanners/service.py`.
- Methods/tools: Python `tempfile.TemporaryDirectory`, `os.walk`, `Path.resolve()`, `shutil.copyfile()`.
- Output: A manifest references a clean copy, rather than scanning an arbitrary working tree in place.

21. Exclude `.git`, excluded paths, symlinks, and out-of-bounds inputs

- Implementation: `build_input_manifest()` and `_is_excluded()` in `src/ai_sentinel/scanners/service.py`.
- Methods/tools: No-follow directory walk; path resolution and containment checks; sorted allowlist; source count/size caps. Current limits are 10,000 files, 5 MB per file, and 25 MB total.
- Output / rule: `.git`, symlinks, excluded paths, oversized files, and paths escaping the root are not passed to scanners.

22. Decision: manifest valid and non-empty?

- Implementation: `build_input_manifest()`; `ScannerInputManifest` in `src/ai_sentinel/scanners/models.py`.
- Validation: Commit SHA is pinned; roots are valid and distinct; staging remains within the intended workspace; files are present, safe, and within limits.
- Yes branch: Continue to scanner coordination at node 24.
- No branch: Node 23 fails retrieval/staging.

23. Fail retrieval/staging; mark assessment/job FAILED with audit event

- Implementation: `GitHubRepositoryScanWorker.execute()` calls `_fail_retrieval()` for `ScannerInputError`; `process_queued_jobs()` and `_fail_execution()` handle uncaught worker failures.
- Methods/tools: Typed errors; repository state re-read and conditional updates; audit/persistence operations.
- Output: Failure is persisted and surfaced; invalid input is not silently treated as an empty successful scan.

Swimlane: Scanner coordinator

24. Set assessment/job RUNNING; record scan-started audit event

- Implementation: `RepositoryScanWorker._execute_manifest()` in `src/ai_sentinel/scanners/service.py`.
- Methods/tools: `model_copy()` state transitions; `AuditService.build_event()`; `write_records_with_audit()`.
- Output: Job stage becomes `scanning`; progress begins at 1; event is `assessment.scan_started`.

25. Select Gitleaks, OSV-Scanner, and Semgrep from assessment options

- Implementation: `_enabled_scanners()` in `src/ai_sentinel/scanners/service.py`.
- Methods/tools: Assessment options map secret scan to Gitleaks, dependency scan to OSV-Scanner, and static analysis to Semgrep. `ScannerKind` is defined in `src/ai_sentinel/scanners/models.py`.
- Output: Ordered tuple; the coordinator runs each enabled scanner sequentially against the same manifest.

26. Decision: enabled scanner count exceeds job tool-call limit?

- Implementation: `RepositoryScanWorker._execute_manifest()`.
- Methods/tools: Compare enabled scanner count to the limit snapshotted in `job.limits.max_tool_calls`.
- Yes branch: Do not invoke scanners; create a failure execution per enabled scanner at node 27.
- No branch: Enter the sequential scanner loop at node 28.

27. Create `tool_call_limit` execution for each enabled scanner

- Implementation: `_error_execution()` in `src/ai_sentinel/scanners/service.py`.
- Methods/tools: `ScannerExecution` records scanner kind, `failure_reason`, empty output, and start/completion timestamps.
- Output: Normalization later marks the test as an error; no Docker process is started.

28. Decision/loop: scanners remain and cancellation not requested?

- Implementation: The `for scanner in enabled_scanners` loop in `_execute_manifest()`; job state is reloaded around each runner call.
- Methods/tools: `Job.cancellation_requested`.
- Yes branch: Build and run the next scanner (node 29 onward).
- No branch: Check the cancellation outcome at node 51 and then finish.

29. Build one ScannerRequest with the same commit-pinned manifest

- Implementation: `_execute_manifest()` and `ScannerRequest` in `src/ai_sentinel/scanners/models.py`.
- Data: Scanner enum plus manifest containing staged root, commit SHA, allowed/excluded paths, timeout, max output bytes, and tool-call limit.
- Safety contract: `read_only=True`; `network_access=False`.

30. Dispatch by scanner kind

- Implementation: `DockerRepositoryScannerRunner.run()` in `src/ai_sentinel/scanners/docker_runner.py`.
- Methods/tools: Dispatch to `DockerGitleaksRunner.run()`, `DockerOsvScannerRunner.run()`, or `DockerSemgrepRunner.run()`.
- Failure: Unknown scanner kinds are rejected rather than sent to an arbitrary executable.

Swimlane: Docker runner - Gitleaks branch

31. Decision: Gitleaks?

- Implementation: `DockerRepositoryScannerRunner.run()`.
- Yes branch: Gitleaks version and scan nodes 32-33.
- No branch: Continue dispatch to the OSV-Scanner decision.

32. Check container version

- Implementation: `DockerGitleaksRunner.run()`; startup check is `DockerGitleaksRunner.validate_startup()`.
- Methods/tools: Run configured pinned image with `gitleaks version`; `_VERSION_PATTERN`; `_run_docker()` with a short timeout and 1 KB output budget.
- Output / rule: Valid version is attached to scan result. Invalid output, nonzero status, timeout, or runner error becomes a structured failure.

33. Scan staged repository for secrets and emit JSON

- Implementation: `DockerGitleaksRunner.run()`.
- Methods/tools: Gitleaks `dir --report-format json --report-path - --exit-code 1 --redact=100 /workspace`.
- Inputs: Read-only staged repository mount at `/workspace`; pinned Gitleaks image.
- Output: Bounded JSON stdout, exit code, version, commit SHA, and timestamps in `ScannerExecution`.

Swimlane: Docker runner - OSV-Scanner branch

34. Decision: OSV-Scanner?

- Implementation: `DockerRepositoryScannerRunner.run()`.
- Yes branch: Nodes 35-42.
- No branch: Semgrep is the remaining configured branch.

35. Find supported npm and Python lockfiles

- Implementation: `DockerOsvScannerRunner._find_lockfiles()` in `src/ai_sentinel/scanners/docker_runner.py`.
- Methods/tools: Match the allowlisted manifest paths against `package-lock.json`, `npm-shrinkwrap.json`, `Pipfile.lock`, `poetry.lock`, `pdm.lock`, `pylock.toml`, `uv.lock`, and `requirements.txt`; resolve within the staged root and build `/workspace/...` arguments.
- Output: Sorted `--lockfile` arguments. `requirements.txt` uses OSV's `requirements.txt:<path>` form.

36. Decision: supported lockfile found?

- Implementation: `DockerOsvScannerRunner.run()` checks the result from `_find_lockfiles()`.
- Yes branch: Validate local advisory databases at node 37.
- No branch: Node 42 returns `no_supported_lockfile`; this is not reported as a clean dependency scan.

37. Validate local npm and PyPI advisory databases

- Implementation: `DockerOsvScannerRunner._validate_database()`.
- Methods/tools: `Path.stat()`, file-size and regular-file checks, modification-time freshness check, `zipfile.is_zipfile()`.
- Inputs: `osv-scanner/npm/all.zip` and `osv-scanner/PyPI/all.zip` beneath `osv_scanner_database_directory`; max age from settings (default 30 days).
- Output: Missing/invalid and stale databases are distinguished.

38. Decision: databases present, valid, and fresh?

- Implementation: `DockerOsvScannerRunner.run()` handles `_MissingVulnerabilityDatabase` and `_StaleVulnerabilityDatabase`.
- Yes branch: Check OSV image version at node 39.
- No branch: Node 41 returns the matching missing/stale database failure.

39. Check container version

- Implementation: `DockerOsvScannerRunner.run()` and `validate_startup()`; `_parse_osv_version()`.
- Methods/tools: Configured OSV container with `--version`, no network, 30-second startup/version limit, 1 KB output cap.
- Output: Parsed version or a structured runner error.

40. Run offline scan with `--no-resolve` and mounted local advisory data

- Implementation: `DockerOsvScannerRunner.run()` and `DockerOsvScannerRunner._container_command()`.
- Methods/tools: OSV-Scanner `scan source --offline --no-resolve --format=json`; allowlisted lockfile args; environment `OSV_SCANNER_LOCAL_DB_CACHE_DIRECTORY=/osv-db`.
- Mounts: Staged repository read-only at `/workspace`; configured local database directory mounted at `/osv-db`; image pull policy `never`.
- Output: Bounded JSON `ScannerExecution`, with pinned commit and version.

41. Return database failure execution

- Implementation: `DockerOsvScannerRunner.run()` and `_execution_failure()` in `src/ai_sentinel/scanners/docker_runner.py`.

- Methods/tools: Failure reason is `missing_vulnerability_database` or `stale_vulnerability_database`.
- Output / rule: No network fallback or pass-shaped result. Adapter later maps the execution to `ERROR`.

42. Return `no_supported_lockfile` execution

- Implementation: `DockerOsvScannerRunner.run()` and `_execution_failure()`.
- Output / rule: Scanner is not launched; normalization marks it not tested rather than pass.

Swimlane: Docker runner - Semgrep branch

43. Stage supported `.py` files only; load bundled rules and fingerprint

- Implementation: `DockerSemgrepRunner.run()`, `_stage_supported_sources()`, and bundled `semgrep_rules.yml`.
- Methods/tools: `importlib.resources.files()`, `resources.as_file()`, temporary directory, `shutil.copyfile()`, SHA-256 from `hashlib.sha256`.
- Validation: Python suffix only; reject absolute, traversal, backslash, symlink, missing, or out-of-root source paths. Rules digest is kept as `config_sha256`.

44. Decision: supported source staged?

- Implementation: `DockerSemgrepRunner.run()` checks `_stage_supported_sources()`.
- Yes branch: Version check at node 46.
- No branch: Node 46 returns `no_supported_source` without starting a scan.

45. Return `no_supported_source` execution

- Implementation: `DockerSemgrepRunner.run()` and `_execution_failure()`.
- Output / rule: Unsupported language coverage is not represented as a clean Semgrep result.

46. Check Semgrep container version

- Implementation: `DockerSemgrepRunner.run()` and `validate_startup()`.
- Methods/tools: Semgrep `--metrics=off --disable-version-check --version`; version regex; bounded `_run_docker()`.
- Output: Valid Semgrep version for provenance or structured runner error.

47. Scan staged sources with bundled rules and emit JSON

- Implementation: `DockerSemgrepRunner.run()`.
- Methods/tools: `semgrep scan --config /semgrep-rules.yml --json --metrics=off --disable-version-check --disable-nosem /workspace`.
- Mounts: Clean staged `.py` workspace read-only; bundled rules mounted read-only at `/semgrep-rules.yml`. Temp HOME/config/cache environment avoids use of repository Semgrep configuration.
- Output: Bounded JSON, version, commit SHA, config SHA-256, timestamps.

Swimlane: Docker runner - shared execution behavior

48. Enforce container isolation and resource limits

- Implementation: `_docker_command()` in `src/ai_sentinel/scanners/docker_runner.py`.
- Tools/settings: Docker `run --rm`; `--network=none`; `--read-only`; `--cap-drop=ALL`; `--security-opt=no-new-privileges`; 512 MB memory; 1 CPU; 128 PIDs; bounded no-exec/no-suid tmpfs at `/tmp`.
- Mounts: Repository is bind-mounted read-only at `/workspace`; optional OSV DB and Semgrep rules mounts are explicit.
- Rule: Scanner containers have no network access; the scanner request protocol also fixes `read_only=True` and `network_access=False`.

49. Return scanner execution or structured timeout/runner failure

- Implementation: `_run_docker()`, `_DockerResult`, `_execution_error()`, and `_execution_failure()` in `src/ai_sentinel/scanners/docker_runner.py`.
- Methods/tools: Python `subprocess.Popen`; stdout reader thread and bounded queue; chunks up to 64 KiB; deadline based on monotonic time; stderr discarded; kill process on timeout/output overrun.
- Output: Exit code, decoded stdout, scanner/version/commit metadata, or a typed failure reason. Failure reasons include timeout, runner error, missing executable/database, stale database, and unsupported input.

50. Record execution and check cancellation before next scanner

- Implementation: `RepositoryScanWorker._execute_manifest()` in `src/ai_sentinel/scanners/service.py`.
- Methods/tools: Append one `ScannerExecution`; reload job; inspect `cancellation_requested`; reject execution whose scanner kind does not match the requested scanner.
- Output: Continue sequentially, or take the partial cancellation route.

Swimlane: Result normalizer

51. Decision: cancellation requested?

- Implementation: Cancellation checks before the loop, after each scanner, and before normal completion in `_execute_manifest()`.
- Yes branch: Normalize only executions already completed at node 52, then persist partial results at node 53.
- No branch: Normalize all expected scanner executions at node 54 onward.

52. Normalize completed executions only (cancellation path)

- Implementation: `ScannerAdapter.normalize()` via the adapter map in `RepositoryScanWorker._finish_cancelled()`.
- Methods/tools: JSON parsing, commit/version/time/output validation, scanner-specific result parsing, redacted `Evidence`, `Finding`, and `TestResult` schema construction.
- Input: Only executions accumulated before cancellation.

53. Persist partial tests/findings/evidence/coverage/audit; set CANCELLED

- Implementation: `RepositoryScanWorker._finish_cancelled()`.
- Methods/tools: Repository operations for tests, findings, evidence, assessment, job, and `assessment.scan_cancelled` audit event.
- Output: Assessment and job become `CANCELLED`; coverage reflects only work completed before the cancellation request.

54. Validate execution metadata, pinned commit, output size, duration, and JSON

- Implementation: `ScannerAdapter.normalize()` and `_execution_error()` in `src/ai_sentinel/scanners/adapters.py`.
- Methods/tools: Validate scanner identity, tool version, expected commit, elapsed duration, permitted exit codes (0/1), UTF-8 output byte limit, valid JSON shape, safe identifiers, paths and package metadata.
- Output / rule: Malformed, mismatched, oversized, or timed-out output is an error, never a pass.

55. Create test results, redacted evidence, and findings for validated matches

- Implementation: `ScannerAdapter.normalize()` and scanner `_parse()` implementations in `src/ai_sentinel/scanners/adapters.py`.
- Methods/tools: `GitleaksAdapter`; `OsvScannerAdapter`; `SemgrepAdapter`; `Evidence`, `Finding`, `FindingProvenance`, and `TestResult` schemas.
- Output: Findings start in `NEEDS_REVIEW`; evidence content is a safe summary and is marked redacted. Tool version, config fingerprint where applicable, commit, test IDs, and source references are retained.
- Scanner-specific parsing: Gitleaks maps RuleID/file/line; OSV maps lockfile/package/advisory/aliases; Semgrep maps check ID/path/line/severity.

56. Map outcomes to PASS, FAIL, ERROR, or NOT_TESTED

- Implementation: `ScannerAdapter.normalize()`; `TestStatus` in `src/ai_sentinel/schemas/test_result.py`.
- Rules: Valid findings -> `FAIL`; valid empty findings -> `PASS`; malformed output, execution failure, invalid metadata -> `ERROR`; unsupported source/lockfile -> `NOT_TESTED`.
- Important: A scanner exit code alone is not enough to call the test a pass; JSON and execution metadata must also validate.

Swimlane: Job store

57. Persist tests, findings, evidence, coverage, and completion audit event

- Implementation: `RepositoryScanWorker._persist_completed()` in `src/ai_sentinel/scanners/service.py`.
- Methods/tools: `RecordOperation` batch through `Repository.write_records_with_audit()`; persistence implementation is JSON or SQL based on settings.
- Output: Tests, findings, and evidence are written with assessment/job transitions and `assessment.scan_completed` event.

58. Decision: any scanner result is ERROR?

- Implementation: `_persist_completed()` checks normalized test results.
- Yes branch: Set assessment status `COMPLETED_WITH_ERRORS`; job status is still `COMPLETED` because the scan workflow finished.
- No branch: Set assessment/job `COMPLETED`.
- Coverage: Error or `NOT_TESTED` means `tested=False`; valid `FAIL` and PASS mean the control was tested, with review/clean explanations.

59. Set assessment to COMPLETED_WITH_ERRORS and job to COMPLETED

- Implementation: `_persist_completed()`.
- Output / rule: Job stage becomes `completed`, progress 100, completion time recorded, and a safe error summary states one or more scanner executions failed.

60. Set assessment and job to COMPLETED

- Implementation: `_persist_completed()`.
- Output: Assessment/job terminal status, completion time, stage `completed`, progress 100, coverage, and scan-completed audit event are persisted together.

Swimlane: Persistent worker (post-scan evaluation)

61. Decision: LLM evaluation requested?

- Implementation: `_run_requested_evaluation()` in `src/ai_sentinel/jobs/worker_cli.py`.
- Methods/tools: `assessment.options.run_llm_evaluations`.
- Yes branch: Node 63 runs the evaluation after repository scan completion.
- No branch: Node 64 skips evaluation; scanner results remain the terminal result of this worker path.

62. Run configured evaluation after scanner results; update evaluation coverage/status

- Implementation: `_run_requested_evaluation()` in `src/ai_sentinel/jobs/worker_cli.py`.
- Methods/tools: `run_inception_suite()` with configured settings and an LLM-token budget bounded by both settings and job snapshot; reload assessment/job; update coverage and evaluation-derived state.
- Output / rule: Evaluation is optional and happens after the scanner worker returns. It does not replace deterministic scanner findings.
- Failure handling: An evaluation run with `ERROR` marks its coverage untested and can change the assessment to `COMPLETED_WITH_ERRORS`; the evaluation run remains separately inspectable.

63. Skip evaluation

- Implementation: Early return in `_run_requested_evaluation()`.
- Condition: `run_llm_evaluations` is false.
- Output: No LLM evaluation is run for the job.

Terminal markers and out-of-scope legend

The diagram's start/end markers are control-flow terminals, not application methods. Its terminal outcomes are: rejected API start; failed retrieval or staging; cancelled after partial work; completed; completed with scanner errors; and worker-level failure surfaced to queue processing. Domain-only checks have their own worker and are not expanded here.

Key data types and configuration

- `ScannerInputManifest`: staged root, commit SHA, sorted allowed files, exclusions, timeout, output-byte cap, tool-call cap.
- `ScannerRequest`: scanner kind + manifest; `read_only=True` and `network_access=False`.
- `ScannerExecution`: scanner kind, version, optional config SHA-256, commit, exit code, stdout, failure reason, timeout flags, timestamps.
- `Settings`: digest-pinned image references; local OSV DB directory and freshness; job duration, tool-call, output, concurrency and LLM budgets.
- Docker resource settings: 512 MB memory, 1 CPU, 128 process limit, 64 MB temporary filesystem; no network, read-only root filesystem, all Linux capabilities dropped, and no-new-privileges.

Source map

- API target/assessment routes: `src/ai_sentinel/api/routes/targets.py`, `src/ai_sentinel/api/routes/assessments.py`.
- Target and assessment rules/queueing: `src/ai_sentinel/targets/service.py`, `src/ai_sentinel/assessments/service.py`, `src/ai_sentinel/policy/authorization.py`, `src/ai_sentinel/jobs/queue.py`.
- Worker queue and CLI: `src/ai_sentinel/jobs/persistent_worker.py`, `src/ai_sentinel/jobs/worker_cli.py`.
- GitHub retrieval: `src/ai_sentinel/github_mcp/adapter.py`.
- Input manifest and scan coordination: `src/ai_sentinel/scanners/service.py`, `src/ai_sentinel/scanners/models.py`.
- Docker and scanner execution: `src/ai_sentinel/scanners/docker_runner.py`, `src/ai_sentinel/scanners/semgrep_rules.yml`.
- Output normalization: `src/ai_sentinel/scanners/adapters.py`.
- Persistence: `src/ai_sentinel/db/repository.py`, `src/ai_sentinel/db/json_repository.py`, `src/ai_sentinel/db/sql_repository.py`.